

Aula Prática 2: Fundamentos de Lógica de Programação

Nicholas Barroso

2026-07-18

Índice

1	Fonte	2
2	Objetivos da Aula	2
3	Fluxograma	4
4	Parte 1 – Tipos de Dados Nativos	5
4.1	Prática no Interpretador	5
5	Parte 2 – Operadores	6
5.1	1. Operadores Aritméticos	6
5.2	2. Operadores Relacionais	6
5.3	3. Operadores Lógicos	7
6	Parte 3 – Listas: Armazenando Múltiplos Valores	7
7	Parte 4 – Estruturas Condicionais (Decisões)	8
7.1	Sintaxe Básica	8
7.2	Exemplo em Bioinformática: Classificação de Genes por Tamanho	8
8	Parte 5 – Laços de Repetição (Loops)	8
8.1	1. Loop <code>for</code> – Iteração em Sequências	9
8.2	2. Loop <code>while</code> – Execução Condicional	9
9	Parte 6 – Aplicação Prática: Cálculo do Conteúdo GC	9
9.1	Problema	9
9.2	Código Comentado	10
9.3	Desafio	10

10 Próximos Passos: Exercícios de Fixação	10
11 Recursos Adicionais	11

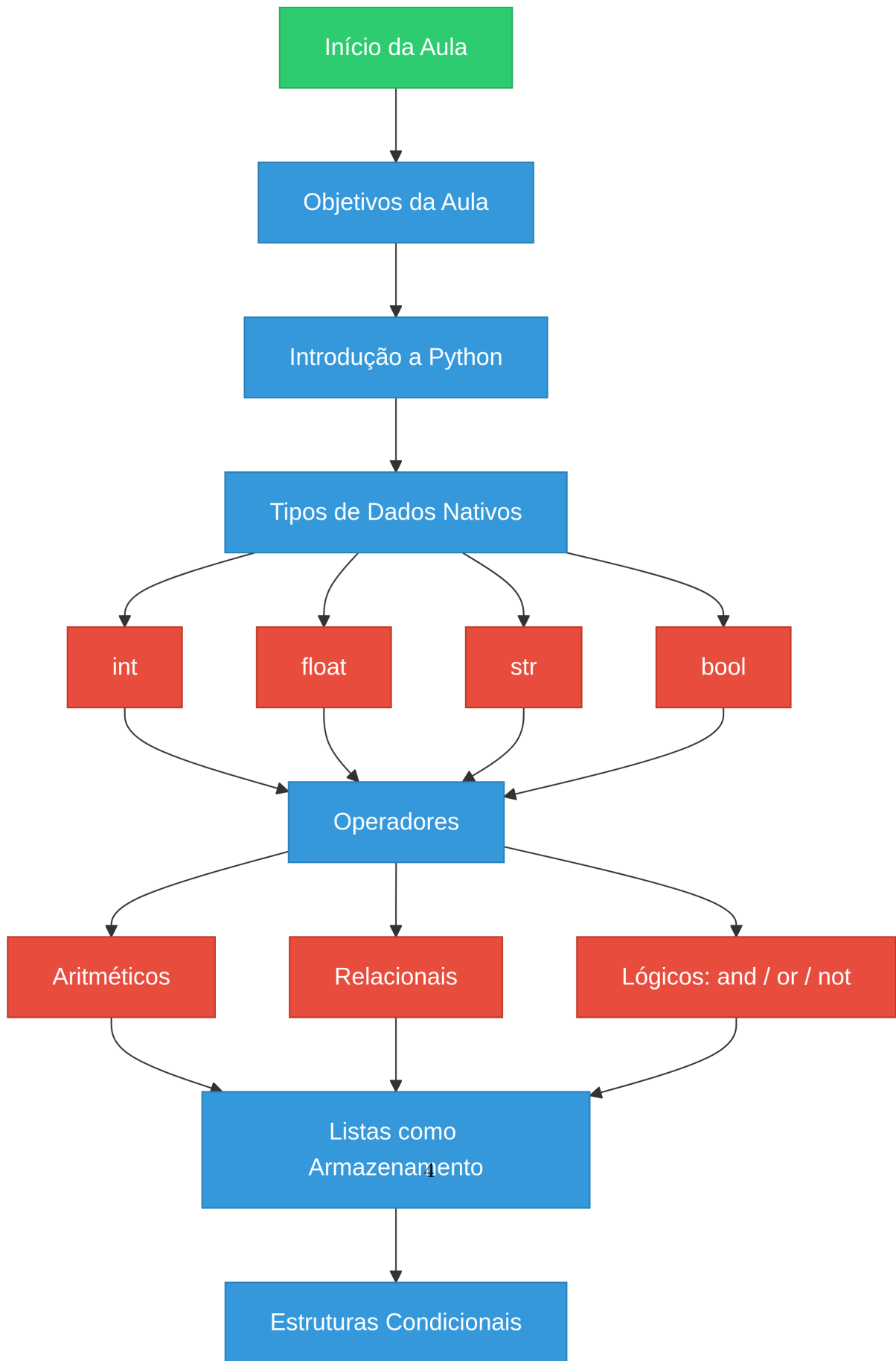
1 Fonte

Documentação oficial da linguagem Python: docs.python.org

2 Objetivos da Aula

- Variáveis;
 - **Operadores** aritméticos, relacionais e lógicos;
 - **Listas** como forma simples de armazenar múltiplos valores;
 - **Raciocínio lógico** com estruturas condicionais (**if/elif/else**);
 - **Raciocínio lógico** com laços de repetição (**for** e **while**);
-

3 Fluxograma



4 Parte 1 – Tipos de Dados Nativos

Objetos tem um **tipo** que define suas características e comportamentos. Conhecer os tipos básicos é essencial para escrever qualquer lógica de programação.

Tipo	Descrição	Exemplo Biológico
Inteiro (int)	Números sem casas decimais	<code>num_genes = 2543</code>
Ponto Flutuante (float)	Números com casas decimais	<code>gc_content = 62.5</code>
Texto/String (str)	Sequência de caracteres	<code>sequencia = "ATGCGTAA"</code>
Booleano (bool)	Valores lógicos Verdadeiro/Falso	<code>eucarioto = True</code>

4.1 Prática no Interpretador

Terminal

Abra o terminal com `Ctrl+Alt+T` e digite `python3` para entrar no interpretador interativo ou digite `idle` e abra o IDLE. Ainda como opção, podemos abrir um arquivo texto, escrever o código e salvar com a extensão `.py`, depois rodar no terminal com `python3 nome_do_arquivo.py`.

Digite os seguintes comandos:

```
# Variáveis com dados biológicos
gene = "lacZ"
num_contigs = 3072
diploide = True
gc_percent = 57.75
```

Agora, use a função `type()` para confirmar os tipos:

```
type(gene)      # <class 'str'>
type(num_contigs) # <class 'int'>
type(diploide)   # <class 'bool'>
type(gc_percent) # <class 'float'>
```

Observação: A linguagem Python faz **tipagem dinâmica** — você não precisa **declarar** o tipo explicitamente; o interpretador infere automaticamente.

5 Parte 2 – Operadores

Operadores são a base de toda decisão e todo cálculo em um programa.

5.1 1. Operadores Aritméticos

Operador	Significado	Exemplo
+	Soma	<code>a + b</code>
-	Subtração	<code>a - b</code>
*	Multiplicação	<code>a * b</code>
/	Divisão (float)	<code>a / b</code>
//	Divisão inteira	<code>a // b</code>
%	Resto da divisão	<code>a % b</code>
**	Potenciação	<code>a ** b</code>

5.2 2. Operadores Relacionais

Comparam dois valores e sempre retornam um `bool` (`True` ou `False`).

Operador	Significado
<code>==</code>	Igual a
<code>!=</code>	Diferente de
<code>></code>	Maior que
<code><</code>	Menor que
<code>>=</code>	Maior ou igual a
<code><=</code>	Menor ou igual a

5.3 3. Operadores Lógicos

Combinam ou invertem valores booleanos — essenciais para condições compostas (ex.: verificar se um valor está *dentro de uma faixa*).

Operador	Significado
and	Verdadeiro se ambas as condições forem verdadeiras
or	Verdadeiro se pelo menos uma condição for verdadeira
not	Inverte o valor lógico

💡 Função print()

A função `print()` exibe informações na tela (STDOUT). Use `help(print)` no interpretador para ver mais detalhes.

6 Parte 3 – Listas: Armazenando Múltiplos Valores

Antes de aprender loops, é preciso saber guardar **vários valores** em um único lugar. A lista é a estrutura mais simples para isso.

- **Características:** ordenada, permite duplicatas, cada valor tem uma posição (índice), começando em 0.
- **Sintaxe:** `genes = ["BRCA1", "TP53", "EGFR"]`

```
genes = ["BRCA1", "TP53", "EGFR"]

print(genes[0])    # "BRCA1"  -> primeiro elemento
print(genes[2])    # "EGFR"   -> terceiro elemento
print(len(genes))  # 3        -> quantidade de elementos
```

```
entradas = [7, -2, 15, 3, 9]  # simula 5 chamadas de input()

for valor in entradas:
    print("Valor lido:", valor)
```

Importante: por enquanto vamos usar listas apenas para **guardar e ler** valores. Não usaremos métodos prontos para trabalhar com listas por enquanto. Nosso objetivo é resolver **na mão**, com `if/for/while`, isso é o que os exercícios de lógica vão pedir.

7 Parte 4 – Estruturas Condicionais (Decisões)

Os condicionais permitem que seu código tome decisões com base em condições.

7.1 Sintaxe Básica

```
if condicao:
    # código executado se condicao for True
elif outra_condicao:
    # código executado se a primeira for False e esta for True
else:
    # código executado se todas as condições anteriores forem False
```

7.2 Exemplo em Bioinformática: Classificação de Genes por Tamanho

```
tamanho_gene = 4500 # em pares de bases

if tamanho_gene > 5000:
    print("Gene muito longo (> 5kb)")
elif 3000 < tamanho_gene <= 5000:
    print("Gene longo (3-5kb)")
elif 1000 < tamanho_gene <= 3000:
    print("Gene médio (1-3kb)")
else:
    print("Gene curto ( 1kb)")
```

8 Parte 5 – Laços de Repetição (Loops)

Loops executam blocos de código repetidamente. São essenciais para processar grandes conjuntos de dados, como sequências de DNA ou listas de leituras de laboratório.

8.1 1. Loop for – Iteração em Sequências

Ideal quando você sabe quantas vezes repetir ou quer percorrer elementos de uma coleção.

```
# Iterando sobre uma lista de genes
genes = ["RuvA", "RuvB", "RuvC"]
for gene in genes:
    print(f"Processando gene: {gene}")
```

```
# Usando range() para repetir um número específico de vezes
for i in range(1, 6): # 1 a 5
    print(f"Ciclo de PCR número {i}")
```

8.2 2. Loop while – Execução Condicional

Executa enquanto uma condição for verdadeira. Ideal quando **não se sabe de antemão** quantas repetições serão necessárias. **Atenção:** sempre atualize a variável de controle para evitar loops infinitos!

```
# Simulando ciclos de PCR
ciclos = 0
while ciclos < 30:
    ciclos += 1 # Incrementa o contador
    print(f"Ciclo {ciclos} concluído")
```

for ou while?

Use **for** quando você já sabe quantas vezes vai repetir (percorrer uma lista, um `range()`).
Use **while** quando a repetição depende de uma condição que só será conhecida durante a execução (ex.: repetir até encontrar algo, repetir até um valor ultrapassar um limite).

9 Parte 6 – Aplicação Prática: Cálculo do Conteúdo GC

9.1 Problema

Calcular a porcentagem de bases Guanina (G) e Citosina (C) em uma sequência de DNA — juntando tipos, operadores, listas/strings, condicionais e loops em um único problema.

9.2 Código Comentado

```
# Inicializa o contador
gc_count = 0

# Sequência de exemplo
sequencia = "ACTGACTGACTGACTG"

# Calcula o tamanho total
tamanho = len(sequencia)

# Percorre cada base da sequência
for base in sequencia.upper(): # .upper() garante letras maiúsculas
    if base == 'G' or base == 'C':
        gc_count += 1

# Calcula a porcentagem
gc_percentual = (gc_count / tamanho) * 100

# Exibe o resultado
print(f"Tamanho da sequência: {tamanho} pb")
print(f"Conteúdo GC: {gc_percentual:.2f}%")
```

9.3 Desafio

Modifique o código para:

1. Aceitar uma sequência fornecida pelo usuário com `input()`;
2. Calcular também o conteúdo de AT (Adenina + Timina);
3. Exibir a proporção GC/AT.

10 Próximos Passos: Exercícios de Fixação

A teoria desta aula é a base necessária para resolver, na sequência:

- [exercicios_fundamentos_logica](#) — 50 exercícios clássicos de lógica (sequência, condicionais, `for`, `while`) usando **apenas** variáveis, operadores, `if/elif/else`, `for` e `while`;

- [exercicios_biotechnologia](#) — os mesmos conceitos aplicados a problemas de biotecnologia (bases nitrogenadas, conteúdo GC, diluições, microplacas, códons).
-

11 Recursos Adicionais

- Documentação oficial: docs.python.org
- Biblioteca Biopython: biopython.org
- Prática online: [Replit](#) ou [Google Colab](#)